

```
from google.colab import drive
drive.mount('/content/drive')
```

```
!pip install torch torchvision numpy opencv-python tqdm
```

[Show hidden output](#)

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import cv2
import numpy as np
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

Using device: cpu

```
class UNet(nn.Module):
    def __init__(self):
        super(UNet, self).__init__()
        def CBR(in_ch, out_ch):
            return nn.Sequential(
                nn.Conv2d(in_ch, out_ch, 3, padding=1),
                nn.BatchNorm2d(out_ch),
                nn.ReLU(inplace=True)
            )
        self.enc1 = CBR(1, 64)
        self.enc2 = CBR(64, 128)
        self.enc3 = CBR(128, 256)
        self.enc4 = CBR(256, 512)
        self.pool = nn.MaxPool2d(2)
        self.center = CBR(512, 1024)
        self.dec4 = CBR(1024 + 512, 512)
        self.dec3 = CBR(512 + 256, 256)
        self.dec2 = CBR(256 + 128, 128)
        self.dec1 = CBR(128 + 64, 64)
        self.final = nn.Conv2d(64, 1, 1)

    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))
        e3 = self.enc3(self.pool(e2))
        e4 = self.enc4(self.pool(e3))
        center = self.center(self.pool(e4))
        d4 = F.interpolate(center, scale_factor=2, mode='bilinear', align_corners=True)
        d4 = self.dec4(torch.cat([d4, e4], dim=1))
        d3 = F.interpolate(d4, scale_factor=2, mode='bilinear', align_corners=True)
        d3 = self.dec3(torch.cat([d3, e3], dim=1))
        d2 = F.interpolate(d3, scale_factor=2, mode='bilinear', align_corners=True)
        d2 = self.dec2(torch.cat([d2, e2], dim=1))
        d1 = F.interpolate(d2, scale_factor=2, mode='bilinear', align_corners=True)
        d1 = self.dec1(torch.cat([d1, e1], dim=1))
        out = torch.sigmoid(self.final(d1))
        return out
```

```
model = UNet()
model.load_state_dict(torch.load('/content/drive/MyDrive/BRISC_UNet.pth', map_location=torch.device('cpu')))
model.to(device)
model.eval()
print("Model loaded on CPU (or GPU if available) and set to evaluation mode.")
```

Model loaded on CPU (or GPU if available) and set to evaluation mode.

```
def dice_coefficient(pred, target, epsilon=1e-6):
    pred = pred.flatten()
    target = target.flatten()
    intersection = (pred * target).sum()
    return (2. * intersection + epsilon) / (pred.sum() + target.sum() + epsilon)
```

```

# calculating dice score on test data
from torch.utils.data import Dataset, DataLoader
import os

class BRISCDataset(Dataset):
    def __init__(self, img_dir, mask_dir):
        self.img_dir = img_dir
        self.mask_dir = mask_dir
        self.images = sorted([f for f in os.listdir(img_dir) if f.endswith('.jpg')])

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img_name = self.images[idx]
        img_path = os.path.join(self.img_dir, img_name)
        mask_name = img_name.replace('.jpg', '.png')
        mask_path = os.path.join(self.mask_dir, mask_name)

        image = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE).astype(np.float32) / 255.0
        mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)
        mask = (mask > 127).astype(np.float32)

        image = cv2.resize(image, (256, 256))
        mask = cv2.resize(mask, (256, 256))
        image = np.expand_dims(image, axis=0)
        mask = np.expand_dims(mask, axis=0)

        return torch.tensor(image), torch.tensor(mask)

test_img_dir = '/content/drive/MyDrive/BRISC/segmentation_task/test/images'
test_mask_dir = '/content/drive/MyDrive/BRISC/segmentation_task/test/masks'

test_dataset = BRISCDataset(test_img_dir, test_mask_dir)
test_loader = DataLoader(test_dataset, batch_size=8, shuffle=False)

dice_scores = []
model.eval()
with torch.no_grad():
    for imgs, masks in test_loader:
        imgs, masks = imgs.to(device), masks.to(device)
        outputs = model(imgs)
        preds = (outputs > 0.5).float()
        dice = dice_coefficient(preds.cpu().numpy(), masks.cpu().numpy())
        dice_scores.append(dice)

print(f"Average Dice Coefficient on test set: {np.mean(dice_scores):.4f}")

```

Average Dice Coefficient on test set: 0.8300

```

# analyzing segmented region
import cv2
import numpy as np
import matplotlib.pyplot as plt
import torch

def predict_mask(image_path, model, device, save_path=None):

    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    resized_img = cv2.resize(image, (256, 256)).astype(np.float32) / 255.0
    input_tensor = torch.tensor(resized_img).unsqueeze(0).unsqueeze(0).to(device)
    model.eval()
    with torch.no_grad():
        output = model(input_tensor)
        pred_mask = (output.squeeze().cpu().numpy() > 0.5).astype(np.uint8) * 255

    fig, axes = plt.subplots(1, 2, figsize=(10, 5))
    axes[0].imshow(image, cmap='gray')
    axes[0].set_title('Original MRI Image')
    axes[0].axis('off')
    axes[1].imshow(pred_mask, cmap='gray')
    axes[1].set_title('Predicted Tumor Mask')
    axes[1].axis('off')
    plt.show()

    if save_path:
        cv2.imwrite(save_path, pred_mask)
        print(f"Predicted mask saved to: {save_path}")

```

```

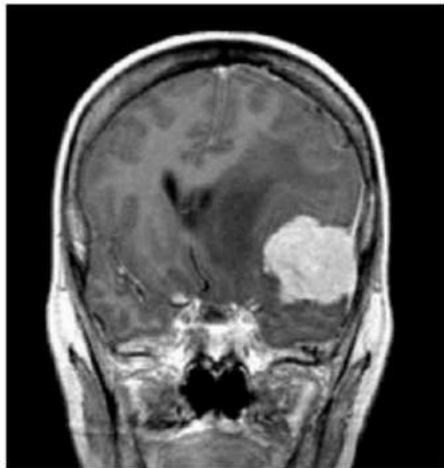
return pred_mask

image_path = '/content/drive/MyDrive/MRI Images/Testing/meningioma/Te-me_0014.jpg'
save_mask_path = '/content/drive/MyDrive/OUTPUT/predicted_maskg18.png'

predicted_mask = predict_mask(image_path, model, device, save_path=save_mask_path)

```

Original MRI Image



Predicted Tumor Mask



Predicted mask saved to: /content/drive/MyDrive/OUTPUT/predicted_maskg18.png

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

```

#Graph CUT TESTING MANUALLY
!pip install PyMaxflow

```

[Show hidden output](#)

Start coding or [generate](#) with AI.

```

import cv2
import numpy as np
import torch
import os
import matplotlib.pyplot as plt

import torch.nn as nn
import torch.nn.functional as F

class UNet(nn.Module):
    def __init__(self):
        super(UNet, self).__init__()
        def CBR(in_ch, out_ch):
            return nn.Sequential(
                nn.Conv2d(in_ch, out_ch, 3, padding=1),
                nn.BatchNorm2d(out_ch),
                nn.ReLU(inplace=True)
            )
        self.enc1 = CBR(1, 64)
        self.enc2 = CBR(64, 128)
        self.enc3 = CBR(128, 256)
        self.enc4 = CBR(256, 512)
        self.pool = nn.MaxPool2d(2)
        self.center = CBR(512, 1024)
        self.dec4 = CBR(1024 + 512, 512)
        self.dec3 = CBR(512 + 256, 256)
        self.dec2 = CBR(256 + 128, 128)
        self.dec1 = CBR(128 + 64, 64)
        self.final = nn.Conv2d(64, 1, 1)

    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))
        e3 = self.enc3(self.pool(e2))

```

```

        e4 = self.enc4(self.pool(e3))
        center = self.center(self.pool(e4))
        d4 = F.interpolate(center, scale_factor=2, mode='bilinear', align_corners=True)
        d4 = self.dec4(torch.cat([d4, e4], dim=1))
        d3 = F.interpolate(d4, scale_factor=2, mode='bilinear', align_corners=True)
        d3 = self.dec3(torch.cat([d3, e3], dim=1))
        d2 = F.interpolate(d3, scale_factor=2, mode='bilinear', align_corners=True)
        d2 = self.dec2(torch.cat([d2, e2], dim=1))
        d1 = F.interpolate(d2, scale_factor=2, mode='bilinear', align_corners=True)
        d1 = self.dec1(torch.cat([d1, e1], dim=1))
        out = torch.sigmoid(self.final(d1))
        return out

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = UNet().to(device)
state = torch.load('/content/drive/MyDrive/BRISC_UNet.pth', map_location=torch.device('cpu'))
model.load_state_dict(state)
model.eval()

def unet_prob_map(image_path, model, device, in_size=256):
    img_gray = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    H, W = img_gray.shape[:2]
    inp = cv2.resize(img_gray, (in_size, in_size)).astype(np.float32) / 255.0
    inp_t = torch.from_numpy(inp)[None, None].to(device)
    with torch.no_grad():
        prob_small = model(inp_t).squeeze().float().cpu().numpy()
    prob = cv2.resize(prob_small, (W, H), interpolation=cv2.INTER_LINEAR).astype(np.float32)
    return img_gray, prob

def grabcut_refine_from_prob(img_gray, prob, iters=5, t_bg=0.2, t_mid=0.5, t_fg=0.8):

    mask = np.full(img_gray.shape, cv2.GC_PR_BGD, dtype=np.uint8)
    sure_bg = prob <= t_bg
    sure_fg = prob >= t_fg
    pr_fg = (prob >= t_mid) & (prob < t_fg)
    pr_bg = (prob > t_bg) & (prob < t_mid)
    mask[sure_bg] = cv2.GC_BGD
    mask[sure_fg] = cv2.GC_FGD
    mask[pr_fg] = cv2.GC_PR_FGD
    mask[pr_bg] = cv2.GC_PR_BGD
    img_bgr = cv2.cvtColor(img_gray, cv2.COLOR_GRAY2BGR)
    bgdModel = np.zeros((1,65), np.float64)
    fgdModel = np.zeros((1,65), np.float64)
    cv2.grabCut(img_bgr, mask, None, bgdModel, fgdModel, iters, mode=cv2.GC_INIT_WITH_MASK)
    out = np.where((mask == cv2.GC_FGD) | (mask == cv2.GC_PR_FGD), 1, 0).astype(np.uint8)
    return (out * 255).astype(np.uint8)

import maxflow

def maxflow_refine_from_prob(img_gray, prob, lam=25.0, sigma=10.0):
    H, W = img_gray.shape
    g = maxflow.Graph[float]()
    nodeids = g.add_grid_nodes((H, W))

    eps = 1e-6
    D_fg = -np.log(np.clip(prob, eps, 1.0)).astype(np.float32)
    D_bg = -np.log(np.clip(1.0 - prob, eps, 1.0)).astype(np.float32)
    g.add_grid_tedges(nodeids, D_bg, D_fg)

    g.add_grid_edges(nodeids, weights=1, structure=None, symmetric=True)

    imgf = img_gray.astype(np.float32)

    diff_h = imgf[:, 1:] - imgf[:, :-1]
    w_h = lam * np.exp(-(diff_h**2) / (2 * sigma**2)).astype(np.float32)

    w_h_full = np.zeros((H, W), dtype=np.float32)
    w_h_full[:, :-1] = w_h
    structure_right = np.array([[0, 0, 0],
                                [0, 0, 1],
                                [0, 0, 0]], dtype=np.int32)
    g.add_grid_edges(nodeids, weights=w_h_full, structure=structure_right, symmetric=True)

    diff_v = imgf[1:, :] - imgf[:-1, :]

```

```

w_v = lam * np.exp(-(diff_v**2) / (2 * sigma**2)).astype(np.float32)
w_v_full = np.zeros((H, W), dtype=np.float32)
w_v_full[:-1, :] = w_v
structure_down = np.array([[0, 0, 0],
                           [0, 0, 0],
                           [0, 1, 0]], dtype=np.int32)
g.add_grid_edges(nodeids, weights=w_v_full, structure=structure_down, symmetric=True)

g.maxflow()
seg = g.get_grid_segments(nodeids)
mask = np.where(seg, 255, 0).astype(np.uint8)
return mask

image_path = '/content/drive/MyDrive/MRI Images/Testing/pituitary/Te-pi_0025.jpg'
save_dir = '/content/drive/MyDrive/OUTPUT'
os.makedirs(save_dir, exist_ok=True)
import os

def infer_category_from_path(image_path):

    cat = os.path.basename(os.path.dirname(image_path))
    return cat

img_gray, prob = unet_prob_map(image_path, model, device, in_size=256)
H, W = prob.shape
pmax = float(prob.max())
fg_frac = float((prob > 0.5).mean())
no_tumor = (pmax < 0.5) or (fg_frac < 0.001)

if no_tumor:

    cv2.imwrite(f'{save_dir}/{base}_orig.png', img_gray)
    plt.figure(figsize=(5,5))
    plt.imshow(img_gray, cmap='gray'); plt.axis('off')
    plt.title('No tumor detected')
    plt.show()
else:

    unet_mask = (prob > 0.5).astype(np.uint8) * 255
    grabcut_mask = grabcut_refine_from_prob(img_gray, prob, iters=5, t_bg=0.2, t_mid=0.5, t_fg=0.8)
    img_gray_d = cv2.medianBlur(img_gray, 3)
    maxflow_mask = maxflow_refine_from_prob(img_gray_d, prob, lam=25.0, sigma=10.0)

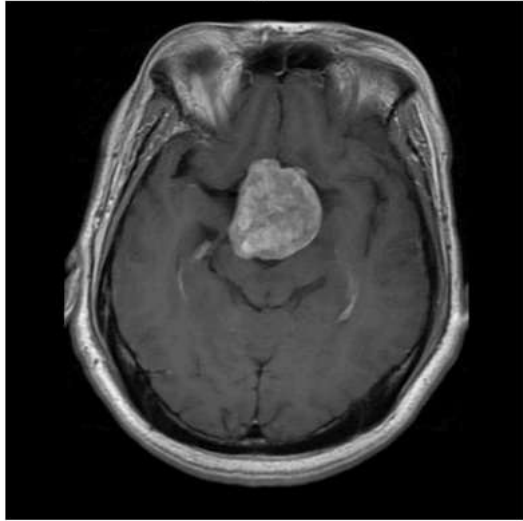
    category = infer_category_from_path(image_path)
    fig, axs = plt.subplots(2,2, figsize=(12,9))
    fig.suptitle(f'Category: {category}', fontsize=14)

    axs[0,0].imshow(img_gray, cmap='gray'); axs[0,0].set_title('Original'); axs[0,0].axis('off')
    axs[0,1].imshow(img_gray, cmap='gray'); axs[0,1].imshow(unet_mask, cmap='Reds', alpha=0.45)
    axs[0,1].set_title('UNet'); axs[0,1].axis('off')
    axs[1,0].imshow(img_gray, cmap='gray'); axs[1,0].imshow(grabcut_mask, cmap='Greens', alpha=0.45)
    axs[1,0].set_title('GrabCut'); axs[1,0].axis('off')
    axs[1,1].imshow(img_gray, cmap='gray'); axs[1,1].imshow(maxflow_mask, cmap='Blues', alpha=0.45)
    axs[1,1].set_title('Max-flow'); axs[1,1].axis('off')
    plt.tight_layout(); plt.show()

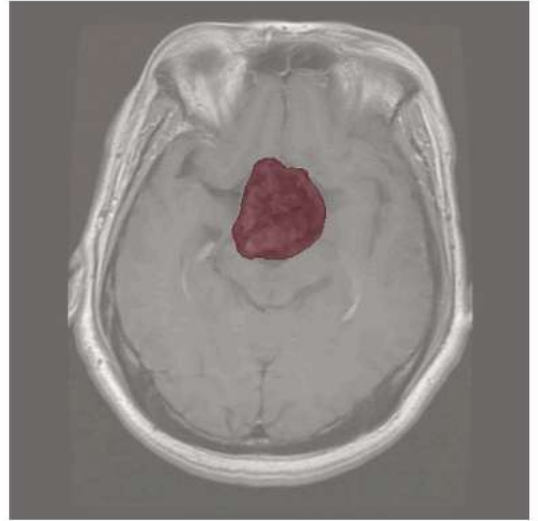
```

Category: pituitary

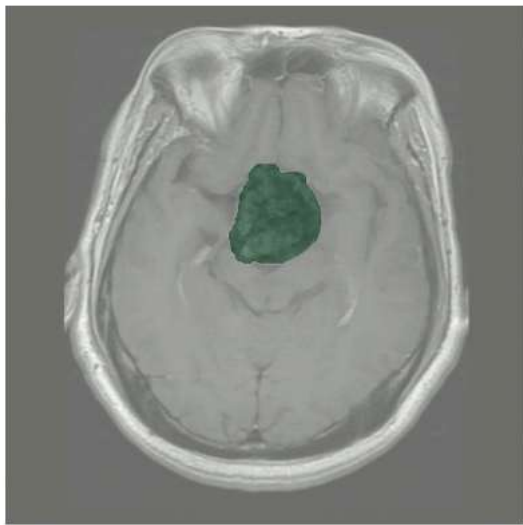
Original



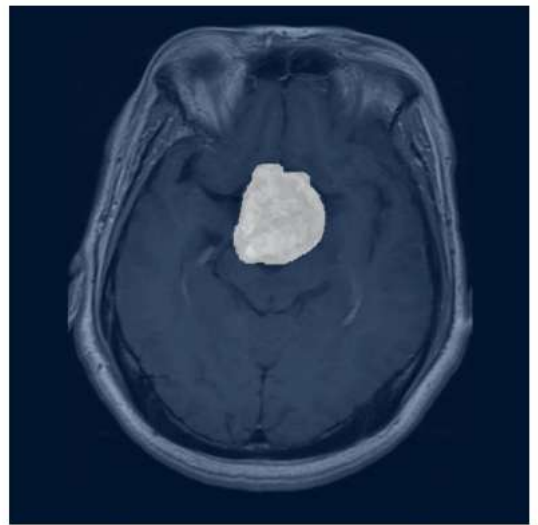
UNet



GrabCut



Max-flow



Start coding or [generate](#) with AI.